

# Reinforcement Learning - Bandits

## Short Intro

Advanced Machine Learning - Master MLDM

Amaury Habrard

`amaury.habrard@univ-st-etienne.fr`

LABORATOIRE HUBERT CURIEN, UMR CNRS 5516

Université Jean Monnet Saint-Étienne

`amaury.habrard@univ-st-etienne.fr`

Semester 1

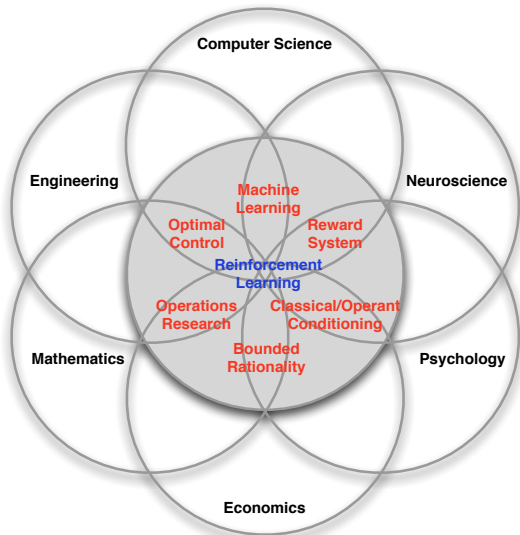
# Outline

- 1 Reinforcement Learning
- 2 Multi-Armed Bandit
- 3 Markov Decision Process (MDP)
- 4 Monte Carlo Tree Search (MCTS)
- 5 Conclusion

- Slides available on moodle
- A small assignment.
- References
  - Many sources available on the Web.
  - Slides on Reinforcement Learning of S. Shalev-Shwartz from which this course is based.  
<http://www.cs.huji.ac.il/~shais/IML2014.html>
  - Foundations of Machine Learning. M. Mohri, A. Rostamizadeh, A. Talwalkar. The MIT Press. 2012.
  - Lectures from Rémi Munos and Aurélien Garivier
  - Davild Silver course <https://www.davidsilver.uk/teaching/>
  - Emma Brunskill course  
<https://web.stanford.edu/class/cs234/>

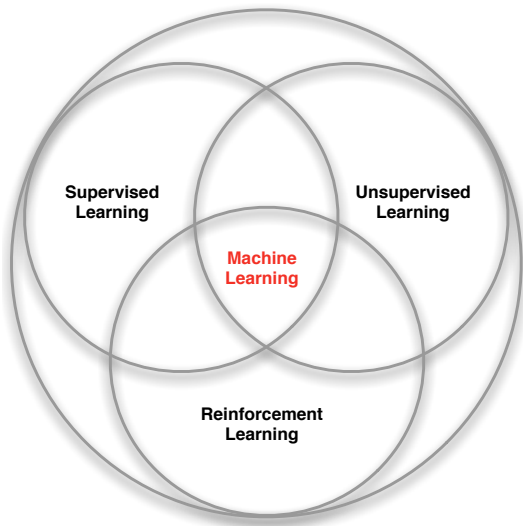
# Reinforcement Learning

# Reinforcement Learning overview 1



From David Silver [https://www.davidsilver.uk/wp-content/uploads/2020/03/intro\\_RL.pdf](https://www.davidsilver.uk/wp-content/uploads/2020/03/intro_RL.pdf)

# Reinforcement Learning overview 2



From David Silver [https://www.davidsilver.uk/wp-content/uploads/2020/03/intro\\_RL.pdf](https://www.davidsilver.uk/wp-content/uploads/2020/03/intro_RL.pdf)

# Presentation

**Goal:** Learn a **policy**, mapping a state space  $S$  to action space  $A$ .

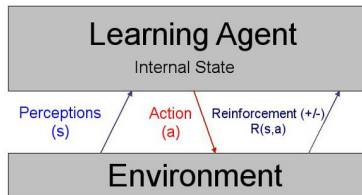
## Learning Process:

For  $t = 1, 2, \dots$

- ① Agent observes state  $s_t \in S$
- ② Agent decides on action  $a_t \in A$  based on the current policy
- ③ Environment provides reward  $r_t \in \mathbb{R}$  after action  $a_t$
- ④ Environment moves the agent to next stage  $s_{t+1} \in S$ .

## Many applications

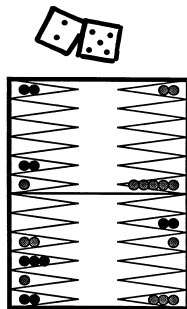
Robotics, Playing games, Finance, Inventory management, ...



# Examples (1/5)

## Backgammon playing

- **Goal:** Learn to play backgammon
- State space is defined by the configuration of playing board (location of each player's states)  
 $\sim 10^{20}$  states
- The actions are the set of legal moves
- Reward is 0 after every turn except when there is a win - reward +1 - or a loss - reward -1





## Examples (2/5)

### Cart pole - inverse pendulum

- **Goal:** Learn to balance the pendulum (pole) in a upright position without hitting the end of the track
- State are the dynamic positions of the cart and pole
- Two actions: move left or move right
- Reward is 0 everywhere except for the states where the pole falls on the cart or the cart hits the end of the track - in these last 2 cases the reward is  $-1$



# Examples (3/5)

## Merge into traffic

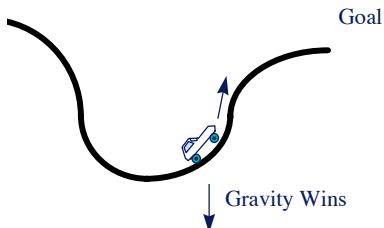
- **Goal:** Adjust the speed of the car according to traffic
- State are composed of positions and velocities of the car and the preceding car
- Action is acceleration/braking command
- Reward is composed of avoiding accidents, smooth driving and making progress



# Examples (4/5)

## Minimum time goal - Car on the Hill

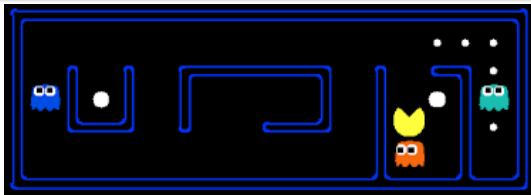
- **Goal:** Generate the shortest path or trajectory to a goal state
- State is car's positions and velocity
- Actions forward thrust, backward thrust or no thrust at all
- Reward is always  $-1$  except for the state goal.  $\rightarrow$  learning the right strategy (needs of momentum/balancing)



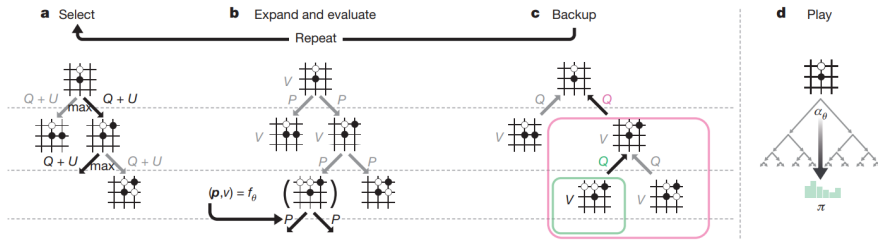
# Examples (5/5)

## Playing Atari Game

<https://www.youtube.com/watch?v=V1eYniJORnk>



# The game of Go

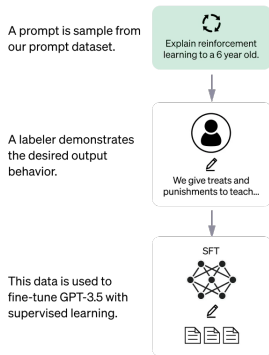


Beyond Human Performance  
D. Silver et al., Nature 2017

# ChatGPT <https://openai.com/index/chatgpt/>

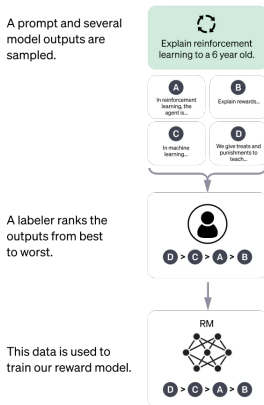
## Step 1

Collect demonstration data and train a supervised policy.



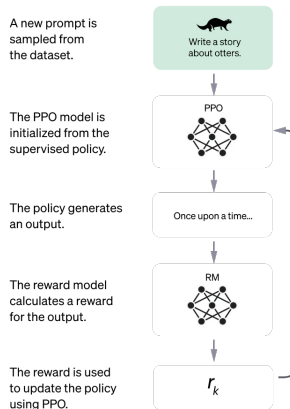
## Step 2

Collect comparison data and train a reward model.



## Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.



# Average Reward and Discounted Reward

We want to maximize a cumulative gain on a long term process

## Average Reward

Given time horizon  $T$ , the average reward of following a policy  $\pi$  is

$$R_T(\pi) = \mathbb{E} \left[ \frac{1}{T} \sum_{t=1}^T r_t \right]$$

## Discounted Reward

Given  $\gamma \in (0, 1)$ , the discounted reward of following a policy  $\pi$  is

$$R_\gamma(\pi) = \mathbb{E} \left[ \sum_{t=1}^{\infty} \gamma^t r_t \right]$$

- discount rate  $\gamma$ : a future gain  $r_{t+k}$  is counted  $\gamma^k$  at time  $t$ .

# Reinforcement Learning and Supervised Learning

## Similarities

*Supervised Learning* is a special case of *Reinforcement Learning* in which  $s_t$  is the “instance” (i.e. example),  $a_t$  is the predicted label,  $-r_t$  is the loss measuring the discrepancy between  $a_t$  and the “true” label  $y_t$ , and  $s_{t+1}$  is chosen independently from  $s_t$  and  $a_t$ .

## Differences:

- In *Supervised Learning*, actions do not affect the environment, therefore we can collect training examples in advance, and only then search for a policy.
- In *Supervised Learning*, the effect of actions is local, while in *Reinforcement Learning* actions have long-term effect.
- In *Supervised Learning*, we are given the correct answer, while in *Reinforcement Learning* we only observe a reward.



# Reinforcement learning

For learning, we have only reinforcement informations, dynamics are random and partially unknown. The objective is to maximize the rewards obtained.

## Environment

- Deterministic or stochastic
- Hostile/Unfriendly (chess game) or not (ex: Tetris)
- Partially observable (robotics)
- Known or unknown

## Reinforcement

- Can reward a sequence of actions
- How to “sacrifice” a short term gain for a better long term gain

# Some issues in Reinforcement Learning

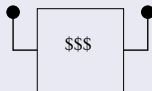
- RL: solve adaptively a problem where the dynamics of the states or the rewards are (partially) unknown. Two possible approaches:
  - **Indirect**: learning a model from the dynamics (kind of supervised learning), then use the model for planification of a strategy
  - **Direct**: direct learning of a strategy of actions (*i.e.* a policy) without any pre-process modeling
- Even when the dynamics are known, the planification problem can be very complex. We are then rather looking for an approximate solution

# Exploration/Exploitation Dilemma

**Exploitation:** acts by maximizing actual knowledge,

**Exploration:** improves actual knowledge

## Example: 2-armed bandit



- At each time step  $t$ , the player chooses an arm  $k$ , receives a reward  $r_t \sim v_k$  where the distributions  $v_k$  (one for each arm) are unknown
- Objective: Maximize  $\sum_t r_t$
- Exemple - rewards previously received: 6, 7, 5 and 4 for the left arm, 5 and 0 for the right one. Which arm should we choose?
- Property: we should never stop exploring, but less and less frequently  $\log(n)/n$
- Different strategies:  $\epsilon$ -greedy, Upper-Confidence-Bounds, bayesian rules, Gibbs sampling, ...

Reinforcement learning: bandit with dynamics on the state.

# Multi-Armed Bandit

# The Multi-Armed Bandit Problem [Robbins, 1952]

- **States:** The state is constant (has no effect)
- **Actions:**  $n$  slot machines (“arms”)
- **Reward:** There exists a deterministic function  $\rho$  from  $A = [n]$  to all distributions over  $[0, 1]$  s.t. for every  $t$ ,  $r_t \sim \rho(a_t)$
- **Notations:**  $\mu_i = \mathbb{E}[r_t | a_t = i]$  expectation of the reward for arm  $i$ ,  
 $i^* = \operatorname{argmax}_i \mu_i$ ,  $\mu^* = \mu_{i^*}$ ,  $\Delta_i = \mu^* - \mu_i$
- **Regret:**

$$\mu^* - \mathbb{E}[R_T(\pi)] = \mu^* - \mathbb{E} \frac{1}{T} \sum_{t=1}^T r_t$$



# The Exploration-Exploitation Tradeoff

## How to pick the next action?

- **Exploitation:** Choose the most promising action based on your current understanding of the problem
- **Exploration:** Maybe there is a better arm ?

## Remark on the expected regret

$$\mu^* - \mathbb{E} \left[ \frac{1}{T} \sum_{t=1}^T r_t \right] = \mathbb{E} \left[ \sum_{i=1}^n T_i(T) (\mu^* - \mu_i) \right]$$

where  $T_i(T) = \frac{1}{T} \sum_{t=1}^T [I_t = i]$  is the empirical average number of times arm  $i$  has been drawn until time  $T$

$\Rightarrow$  a good bandit algorithm should draw few suboptimal arms.

# Naive approach: first explore then exploit

## Procedure

- Pure exploration for the first  $m$  iterations (pick actions at random)
- Let  $\hat{i} = \operatorname{argmax}_i \hat{\mu}_i$  where  $\hat{\mu}_i = \text{empirical average}(r_t | a_t = i)$
- Pure exploitation for the rest of the  $T - m$  iterations (*i.e.* always pick  $\hat{\mu}_{\hat{i}}$ )

## Analysis

- **Claim:** If  $m$  is in order of  $n \log(n) / \epsilon^2$  then for all  $i$ ,  $|\mu_i - \hat{\mu}_i| \leq \epsilon$
- Proof: Hoeffding + union bound

# Hoeffding's inequality

## Hoeffding bound

Let  $Z_1, \dots, Z_m$  be a sequence of *i.i.d* random variables and let  $\bar{Z} = \frac{1}{m} \sum_{j=1}^m Z_j$ . Assume that  $\mathbb{E}[\bar{Z}] = \mu$  and for all  $1 \leq i \leq m$  we have  $a \leq Z_j \leq b$ . Then, for any  $\epsilon > 0$ ,

$$P \left[ \left| \frac{1}{m} \sum_{j=1}^m Z_j - \mu \right| > \epsilon \right] \leq 2 \exp(-2m\epsilon^2/(b-a)^2).$$

- We have  $r_t \in [0, 1]$ , thus  $a$  refers to 0 and  $b$  to 1
- The bound says that with proba at most  $2 \exp(-2m\epsilon^2)$  we have  $|\mu_i - \hat{\mu}_i| \geq \epsilon$  for one arm  $i$ .
- Fix  $\delta/2n = 2 \exp(-2m\epsilon^2)$ , we have  $m = \log(2n/\delta)/2\epsilon^2$ . Taking  $\delta = n^{-(2n-1)}$  validates the assumption ( $m = (2n-2) \log(2n)/2\epsilon$ )
- By proba  $1 - \delta/2$  we have that  $\forall i, |\mu_i - \hat{\mu}_i| \leq \epsilon$



# Naive approach: first explore then exploit

Procedure - We assume the rewards to be in  $[0, 1]$

- Pure exploration for the first  $m$  iterations (pick actions at random)
- Let  $\hat{i} = \operatorname{argmax}_i \hat{\mu}_i$  where  $\hat{\mu}_i = \text{empirical average}(r_t | a_t = i)$
- Pure exploitation for the rest of the  $T - m$  iterations (*i.e.* always pick  $\hat{\mu}_i$ )

# Naive approach: first explore then exploit (2)

## Analysis

- **Claim:** If  $m$  is in order of  $n \log(n)/\epsilon^2$  then for all  $i$ ,  $|\mu_i - \hat{\mu}_i| \leq \epsilon$
- Regret:

$$\begin{aligned}
 \mu^* - \frac{m\bar{\mu} + (T - m)\mu_{\hat{i}}}{T} &= (\mu^* - \mu_{\hat{i}}) + \frac{m}{T}(\mu_{\hat{i}} - \bar{\mu}) \\
 &\leq (\mu^* - \hat{\mu}_{i^*} + \hat{\mu}_{i^*} - \hat{\mu}_{\hat{i}} + \hat{\mu}_{\hat{i}} - \mu_{\hat{i}}) + \frac{m}{T} [\text{since } \hat{\mu}_{\hat{i}} > \bar{\mu}, \text{ thus } \hat{\mu}_{\hat{i}} - \bar{\mu} \in [0, 1] ] \\
 &\leq (\mu^* - \hat{\mu}_{i^*} + \hat{\mu}_{i^*} - \hat{\mu}_{i^*} + \hat{\mu}_{\hat{i}} - \mu_{\hat{i}}) + \frac{m}{T} [\text{since if } i^* \neq \hat{i}, \hat{\mu}_{i^*} < \hat{\mu}_{\hat{i}}] \\
 &\leq |\mu^* - \hat{\mu}_{i^*}| + 0 + |\hat{\mu}_{\hat{i}} - \mu_{\hat{i}}| + \frac{m}{T} \leq 2\epsilon + \frac{n \log(n)}{T\epsilon^2}
 \end{aligned}$$

- For the best  $\epsilon$ , the regret is of order of  $\left(\frac{n \log(n)}{T}\right)^{1/3}$

## EXP3 [Auer, Cesa-Bianchi, Freud, Schapire]

EXP3: **Exploration-Exploitation** using **Exponentiated Gradient**

- a type of  **$\epsilon$ -greedy exploration**
- Idea: Set arm  $i$  with probability

$$p(i) = (1 - \epsilon)w(i) + \epsilon \frac{1}{n}$$

- The probabilities are updated at each iteration in the form of a weight vector  $w_t$ , such that

$$\mathbf{p} = (1 - \epsilon)\mathbf{w}_t + \epsilon \mathbf{1}/n$$

- $\Rightarrow$  explore with proba  $\epsilon$  and exploit with proba  $(1 - \epsilon)$  but pick the arm according to  $\mathbf{p} = \mathbf{w}_t$ ,
- update is made using Stochastic Gradient with exponential weight

## EXP3 [Auer, Cesa-Bianchi, Freud, Schapire] (2)

## Algo

- Initialize:  $w_{(1)} = (1/n, \dots, 1/n)$
- At each step  $t$ , the player chooses the arm  $I_t \sim p_t$  where

$$p_t(i) = (1 - \beta) \frac{w_t(i)}{\sum_{k=1}^K w_t(k)} + \frac{\beta}{n}$$

- Let  $\hat{r}_t(i) = [I_t = i]r_t(i)/p_t(i)$ , with  $r_t(i)$  the reward received
- Update:  $w^{(t+1)}(i) = \frac{1}{Z_t} w_i^{(t)} \exp(-\eta \hat{r}_t(i))$ , with  $Z_t$  a normalization factor.
- The update makes sure that we have some exploration (we never completely move components of  $w$  to zero)
- Regret analysis: it can be shown to be of order  $\left(\frac{n \log(n)}{T}\right)^{1/2}$

# Upper Confidence Bound (UCB)

## Optimism in the face of uncertainty [Lai and Robbins, 1985]

- Estimate an upper confidence  $\hat{U}_t(i)$  for each action consisting of using arm  $i$
- such that  $\mu_i \leq \hat{\mu}_i + \hat{U}_t(i)$
- This depends on the number of times  $N_i(t)$  the arm  $i$  has been selected
  - small  $N_i(t) \Rightarrow$  large  $\hat{U}_t(i)$  (estimated value is uncertain)
  - large  $N_i(t) \Rightarrow$  small  $\hat{U}_t(i)$  (estimated value is rather accurate)
- Select arm (or action) maximizing Upper Confidence Bound (UCB)

$$i = \operatorname{argmax}_i \hat{\mu}_i + \hat{U}_t(i)$$

# Upper Confidence Bound (UCB)

- From Hoeffding inequality, if we pull arm  $i$ :

$$P[\mu > \hat{\mu}_i + \hat{U}_t(i)] \leq e^{-2N_t(i)U_t(i)^2}$$

- Fixing  $\delta = e^{-2N_t(i)U_t(i)^2}$ , we get

$$U_t(i) = \sqrt{\frac{-\log p}{2N_t(i)}} = \sqrt{\frac{\log(1/p)}{2N_t(i)}}$$

- Reduce  $\delta$  as we observe more rewards, e.g.:  $\delta = t^{-4}$ , we obtain

$$U_t(i) = \sqrt{\frac{2 \log t}{N_t(i)}}, \text{ ensures to select the optimal arm as } t \rightarrow \infty.$$

- Then, we get that if we pulled arm  $i$  for  $N_i(t)$  times then:

$$\mu_i \leq \hat{\mu}_i + \sqrt{\frac{2 \log(t)}{N_i(t)}} := \text{UCB}_i(t)$$

# Result for UCB

The UCB rule is to pull the arm that maximizes  $UCB_i(t)$ :

$$i_t = \operatorname{argmax}_i \hat{\mu}_i + \sqrt{\frac{2 \log(t)}{N_i(t)}}$$

## Proposition

- Each sub-optimal arm  $i$  is drawn at most

$$\mathbb{E}[N_i(t)] \leq 8 \frac{\log(t)}{\Delta_i^2} + 2$$

- The cumulated regret of UCB with  $m$  arms is bounded by

$$\mathbb{E}[R_T] = \sum_i \Delta_i \mathbb{E}[N_i(t)] \leq 8 \sum_{i: \Delta_i > 0} \frac{\log T}{\Delta_i} + 2m$$

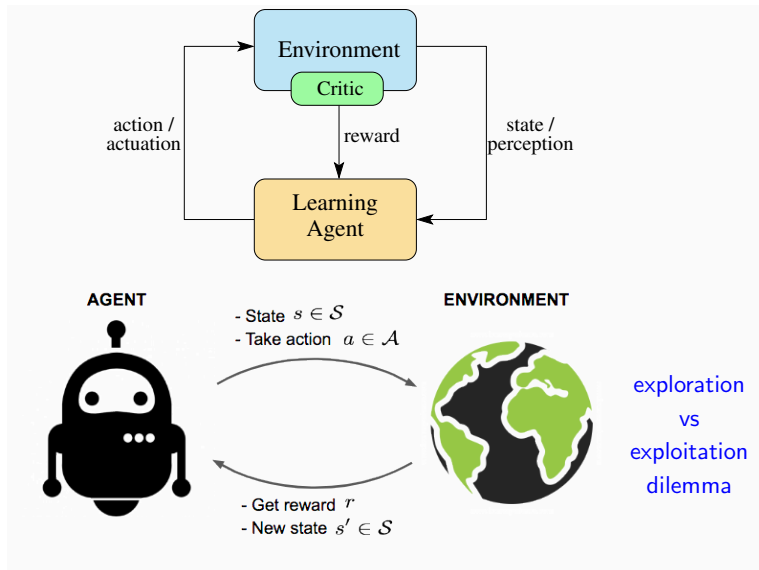
# Other approaches

- Improvements by optimizing the bounds of the confidence intervals
- Application to online learning
- Application in a Markov Decision Process
- Adversarial bandits
- For games: optimizing the Nash equilibrium
- ....



# Markov Decision Process

# A Model for RL: MDP



# Markov Process

## Markov chain

Dynamic system with discrete time  $(s_t)_{t \in \mathbb{N}} \in S$  where  $S$  is the state space such that  $P(s_{t+1} = s | s_t, s_{t-1}, \dots, s_0) = P(s_{t+1} = s | s_t)$ : all the relevant information for prediction in the future is included in the current state. A Markov chain defined over  $S$  is defined by an initial state  $s_0$  and transition probabilities:  $p(y|x) = P(s_{t+1} = y | s_t = x)$

A Markov Decision Process (MDP) defines a model of the environment and interactions with environment widely adopted in reinforcement learning.

# Markov Decision Process (MDP)

## Quadruplet $(S, A, \tau, r)$

- $S$ : state space (assumed to be finite, but can be countable or continuous)
- $A$ : action space
- $\tau$ : a transition probability - for every  $t$ ,  $s_{t+1} \sim \tau(s_t, a_t)$  where  $\tau$  is a deterministic function over  $S \times A$   
if  $\tau$  is stochastic the probability of the event is  $p(s_{t+1}|s_t, a_t)$ ,
- $\rho$ : a reward probability - for every  $t$ ,  $r_t$  is a random variable over  $[0, 1]$  whose distribution depends deterministically only on  $(s_t, a_t)$  and we denote its expected value by  $\rho(s_t, a_t)$ ,  
also denoted as  $r(s_{t+1}, a_t, s_t)$ : reward received for the transition from  $s_t$  to  $s_{t+1}$  with action  $a_t$ .
- Some definition include a start state  $s_0$  (we have thus a quintuplet).
- The Markov property implies that  $[s_{t+1}, r_t]$  is conditionally independent of  $(s_{t-1}, a_{t-1}), (s_{t-2}, a_{t-2}), \dots, (s_1, a_1)$  given  $(s_t, a_t)$ .

# Controlled Markov Process

- Initial State  $s_0$
- For time  $t = 1, 2, \dots$ 
  - choose action  $a_t$  according to a policy  $\pi_t$
  - get reward  $r_t \sim \rho(s_t, a_t)$
  - switch to new stage  $s_{t+1} \sim \tau(s_t, a_t)$

**Cumulated reward:**  $R = \sum_{t=0}^{\infty} \gamma^t r_t$ ,  $\gamma \in (0, 1)$  a discount parameter

**Choose the actions so as to maximize the cumulated reward in expectation**

# Policy - a mapping $\pi : S \rightarrow A$

## Decision rule: $\pi_t$

$\pi_t$  determines at each step  $t$  an action (deterministically or stochastically) for every state:

- deterministic:  $\pi_t : X \rightarrow A, \pi_t(s) = \text{action chosen in state } s$
- stochastic:  $\pi_t : S \times A \rightarrow \mathbb{R}, \pi_t(a|s) = \text{proba to choose action } a \text{ in state } s$
- can be non stationary, but actually deterministic stationary policies can do as well.

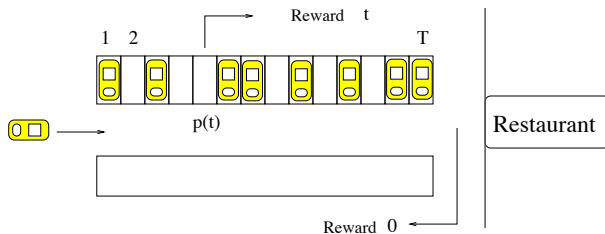
## Policy, strategy or plan

Sequence of decision rules  $\pi = (\pi_0, \pi_1, \pi_2, \dots)$ .

If  $\pi$  is stationary with time, this is a stationary policy or Markovian policy ( $\pi = (\pi, \pi, \pi, \dots)$ )

and the sequence of rewards is a hidden Markov chain

# Example Parking

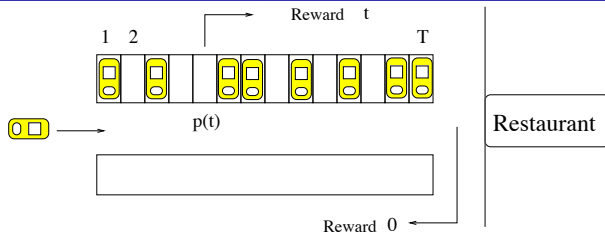


A driver wants to park his car as close as possible to the restaurant.

At each step 2 actions: stop (and park) or continue

- The driver can see if a place is free only when he is in front of it, he decides then to park or to continue.
- Each place  $i$  is free with proba  $p(i)$
- place  $i$  gives reward  $i$ , if not parked reward is 0.

# Example Parking



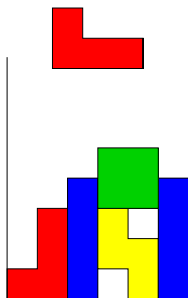
A driver wants to park his car as close as possible to the restaurant.

At each step 2 actions: stop (and park) or continue

- State space:  $\{(P_i, S_i)\} \cup \{\text{parked, left}\}$ ,  $S_i \in \{\text{Taken, Available}\}$ , if the car is parked he reaches state parked, other if he never parks after the  $P$  places he reaches left - both states are terminal.
- Actions:  $\{\text{park, continue}\}$ , if state is of the form  $(\cdot, \text{Taken})$ , the only action available is continue
- Rewards: for any  $i \in [1, P]$ ,  $r(x, \text{park}, \text{parked}) = i$ , 0 otherwise
- Next state: a place  $o$  is available with proba  $p(i)$ ; then the driver makes its choice.



# Example Tetris



- State: Wall configuration + new piece
- Actions: possible positions of the new piece
- Rewards: number of lines removed
- Next state: new wall configuration + random for the new piece

- We can show that for every strategy, the game finishes with probability 1, the expectation of the sum of the rewards is thus finite.
- Problem: the state space is very large ( $10^{61}$  for 20 lines of width 10 and 7 pieces different)

# Example: Inverted pendulum

- State: horizontal position, angular position and velocity  
State space:  $s = [0, 1] \times [-\pi, \pi] \times \mathbb{R}$  ( $\pi$  is here the constant 3.1416...)
- Actions: move left or right Action space  $A = \{-1, 1\}$
- Rewards: proportional to the height of the stick end  
if  $s_t(x_t, \theta_t, \phi_t)$ ,  $r_t = \sin(\theta_t)$
- Next state transition: given by the laws of physics

# Example: Retail Store Management (From A. Garivier) 1/2

You owe a bike store. During week  $t$ , the (random) demand is  $d_t$  **units**. On Monday morning you may choose to command  $a_t$  additional units: they are delivered immediately before the shop opens.

For each week:

- Maintenance cost:  $h(s)$  for  $s$  units in stock left from the previous week
- Command cost:  $C(a)$  for  $a$  units
- Sales profit:  $f(q)$  for  $q$  units sold
- Constraints:
  - your warehouse has a maximal capacity of  $M$  units (any additional bike gets lost)
  - you cannot sell bikes that you do not have in stock

# Example: Retail Store Management (From A. Garivier) 2/2

- State: number of bikes in stock on Sunday  
State space=  $S = \{0, \dots, M\}$
- Action: number of bikes ordered at the beginning of the week  
Action space:  $A = \{0, \dots, M\}$
- Reward = balance of the week: if you order  $a_t$  bikes:  

$$r_t = -C(a_t) - h(s_t) + f(\min(d_t, s_t + a_t, M))$$
 the *min* here ensures that we cannot sell more bikes than we have, ideally this should be  $d_t$  except if we do not have enough bikes.
- Transition: you end the week with  

$$s_{t+1} = \max(0, \min(M, s_t + a_t) - d_t) \text{ bikes}$$
 We may assume for example that  $h(s) = h \cdot s$ ,  $f(q) = p \cdot q$  and  

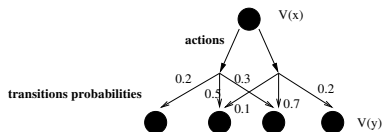
$$C(a) = c_0 \cdot \mathbb{1}_{\{a>0\}} + c \cdot a$$

# MDP - algorithms

- **Value Iteration:** Find the optimal policy when  $\tau$  and  $\rho$  are known
- **Q-Learning:** Find the optimal policy when  $\tau$  and  $\rho$  are not known

# Value function $V(x)$

- Associate a value to each state: models what the agent can expect in terms of best we can hope on average in this state
- The value  $V(x)$  in a state depends on the immediate reward and on the value of the resulting states  $V(y)$ .



- The value in a state must correspond to the immediate reward plus the average value of the next state if I choose the optimal action  

$$V(x) = \max_a [r(x, a) + \mathbb{E}[V(Y)|x, a]]$$
 - **Bellman equation**
- If one knows  $V$ : no surprise “on average”. We learn  $V$  with the temporal differences.
- When  $V$  is known, we can choose the best action at each state such that maximizing the function locally corresponds to maximizing the reinforcement in a long term perspective.

# Criterion to optimize - Value function - performance measure

The value  $V^\pi(s)$  of a policy  $\pi$  at state  $x \in S$  is defined as the expected reward returned when starting at  $x$  and following  $\pi$

- Finite temporal Horizon

$V^\pi(t, x) = \mathbb{E}[\sum_{s=t}^{T-1} r(x_s, \pi(x_s)) + R(x_T) | x_t = x; \pi]$  where  $T$  is a terminal reward function (terminal state).

- Infinite temporal horizon with discount factor (for future rewards)

$V^\pi(x) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r(x_t, \pi(x_t)) | x_0 = x; \pi]$  where  $\gamma$  is a discount factor

- Infinite temporal horizon without discount factor

$V^\pi(x) = \mathbb{E}[\sum_{t=0}^T r(x_t, \pi(x_t)) | x_0 = x; \pi]$  where  $T$  is the first time step reaching a final state (absorbing state).

- Infinite temporal horizon with average criterion

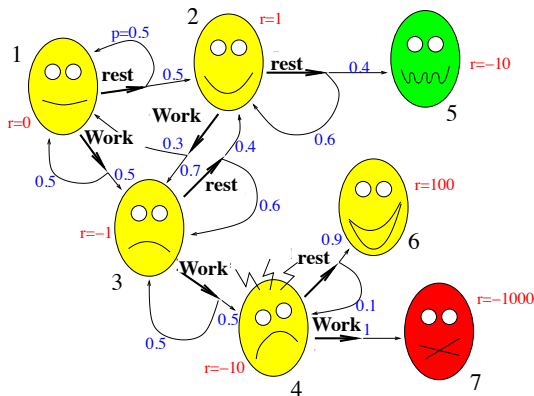
$V^\pi(x) = \lim_{T \rightarrow \infty} \frac{1}{T} \mathbb{E}[\sum_{t=0}^{T-1} r(x_t, \pi(x_t)) | x_0 = x; \pi]$

# A word on the discount factor

- Mathematically convenient (avoid infinite returns and values)
- Humans often act as if there is a discount factor  $\gamma < 1$
- $\gamma = 0$ : Only care about immediate reward
- $\gamma = 1$ : Future reward is as beneficial as immediate reward
- If episode lengths are always finite ( $T < \infty$ ), one can use  $\gamma = 1$

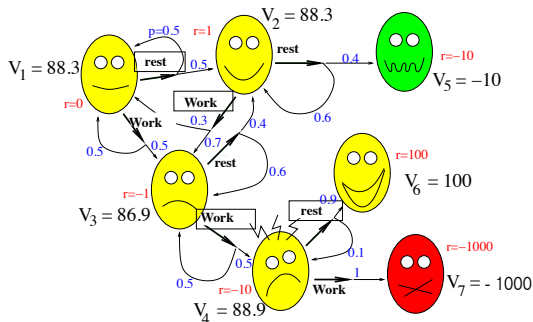


# Example - Student Dilemma



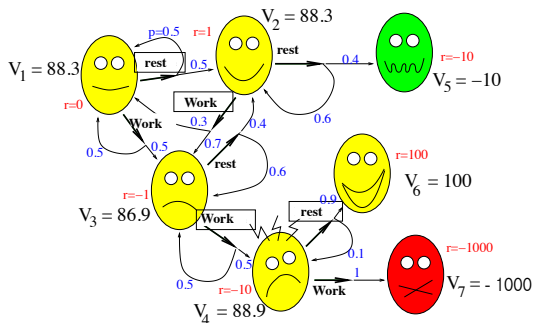
- States 5, 6 et 7 are final states (rewards in states are the same for each action)
- Goal: maximize the sum of rewards until reaching a final state
- We assume the student knows the probabilities of transitions and award functions - How to solve this problem?

# Example - Student Dilemma - Solution



- We have:  $V_5 = -10$ ,  $V_6 = 100$ ,  $V_7 = -1000$ .
- $V_4 = \max\{-10 + 0.9 * V_6 + 0.1 * V_7; -10 + 1 * V_7\} = -10 + 0.9 * V_6 + 0.1 * V_7 \simeq 88.9$ ,
- $V_3 = \max\{-1 + 0.4 * V_2 + 0.6 * V_3; -1 + 0.5 * V_4 + 0.5 * V_3\}$

# Example - Student Dilemma - Solution (2)



- $V_2 = \max\{1 + 0.4 * V_5 + 0.6 * V_2; 1 + 0.7 * V_3 + 0.3 * V_1\}$  and  $V_1 = \max\{0.5 * V_2 + 0.5 * V_1; 0.5 * V_3 + 0.5 * V_1\}$ ,
- We can deduce that  $V_1 = V_2 = 88.3$  (need to check all cases)
- And thus  $V_3 = -1 + 0.5 * V_4 + 0.5 * V_3 \simeq 86.9$ .
- Then in state 1 we rest, in state 2 we work, in state 3 we work and in state 4 we rest.

## Example - Student Dilemma - Solution (3)

- Now, assuming that the student does not know transition probabilities and rewards, can he learn to solve this problem?  
⇒ this is the objective of Reinforcement learning.

# Value function with finite horizon $T$

- For a given policy  $\pi = (\pi_0, \dots, \pi_{T-1})$ , the expected reward starting from  $x$  at time  $t$  is defined by the following *Bellman equation*

$$V^\pi(t, x) = \mathbb{E}\left[\sum_{s=t}^{T-1} r(x_s, \pi_s(x_s)) + R(x_T) \mid x_t = x; \pi\right].$$

- The **optimal value function** is  $V^*(t, x) = \max_{\pi} V^\pi(t, x)$
- A policy  $\pi^*$  is optimal if  $V^{\pi^*}(t, x) = V^*(t, x)$

**Proposition**  $V^*(t, x)$  is the solution of the Bellman equation

- $V^*(t, x) = \max_{a \in A} [r(x, a) + \sum_{y \in S} p(y|x, a) V^*(t+1, y)]$  for  $0 \leq t < T$
- $V^*(T, x) = R(x)$ : reward at a final state at time  $T$
- The policy defined as follows is optimal for  $0 \leq t < T$ :  
 $\pi_t^*(x) \in \arg \max_{a \in A} [r(x, a) + \sum_{y \in S} p(y|x, a) V^*(t+1, y)]$

# Proof

- We have  $V^*(T, x) = R(x)$ . We solve  $V^*$  by back propagation for  $t < T$
- Every policy  $\pi = (\pi_t, \pi_{t+1}, \dots, \pi_{T-1})$  from the initial state  $x$  at time  $t$  is of the form  $\pi = (a, \pi')$  with  $a \in A$  and  $\pi' = (\pi_{t+1}, \dots, \pi_{T-1})$ .

$$\text{Thus, } V^*(t, x) = \max_{\pi} \mathbb{E} \left[ \sum_{s=t}^{T-1} r(x_s, \pi_s(x_s)) + R(x_T) \mid x_t = x; \pi \right]$$

$$\begin{aligned} &= \max_{a, \pi'} \left[ r(x, a) + \sum_{y \in S} p(y|x, a) V^{\pi'}(t+1, y) \right] \\ &= \max_{a \in A} \left[ r(x, a) + \sum_{y \in S} p(y|x, a) \max_{\pi'} V^{\pi'}(t+1, y) \right] \\ &= \max_{a \in A} \left[ r(x, a) + \sum_{y \in S} p(y|x, a) V^*(t+1, y) \right]. \end{aligned}$$

# Proof continued

- Equality above is justified by:

- Trivial inequality:

$$\sum_y p(y|x, a) V^{\pi'}(t+1, y) \leq \sum_y p(y|x, a) \max_{\pi'} V^{\pi'}(t+1, y)$$

- Let  $\bar{\pi} = (\bar{\pi}_{t+1}, \dots)$  a policy such that  $\bar{\pi}_y = \arg \max_{\pi'} V^{\pi'}(t+1, y)$ , thus

$$\sum_y p(y|x, a) \max_{\pi'} V^{\pi'}(t+1, y) = \sum_y p(y|x, a) V^{\bar{\pi}}(t+1, y) \leq \max_{\pi'} \sum_y p(y|x, a) V^{\pi'}(y)$$

- $\pi_t^*$  corresponds to the max at each iteration, by back-propagation w.r.t.  $t$   $V^* = V^{\pi^*}$

# Infinite temporal horizon and adaptive criterion

- For a given policy  $\pi = (\pi_0, \dots, \pi_{T-1})$ , the value function from  $x$ :

$$V^\pi(x) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t r(x_t, \pi_t(x_t)) \mid x_0 = x; \pi\right].$$

with  $0 \leq \gamma < 1$  is an update parameter guaranteeing the convergence of the series

- The **optimal value function** is  $V^* = \sup_{\pi} V^\pi$

**Proposition**  $V^*(t, x)$  is the solution of the Bellman equation

- $V^*(t, x) = \max_{a \in A} [r(x, a) + \sum_{y \in X} \gamma p(y|x, a) V^*(t+1, y)]$  for  $0 \leq t < T$
- $V^*(T, x) = R(x)$
- The policy defined as follows is optimal:  
 $V^*(x) = \max_{a \in A} [r(x, a) + \sum_{y \in X} \gamma p(y|x, a) V^*(y)],$  for  $0 \leq t < T$

Proof shares similar ideas with the previous proof.



# A note on Bellman equations

The Bellman equations can be rewritten as (with an optional discount factor  $\gamma \in [0, 1)$ ):

$$\mathbf{V} = \mathbf{R} + \gamma \mathbf{P} \mathbf{V}$$

- $\mathbf{P}$  is the transition probability matrix where  $\mathbf{P}_{s,s'} = Pr[s'|s, \pi(s)]$
- $\mathbf{R}$  is the reward column matrix whose  $s^{th}$  component is  $E[r(s, \pi(s))]$
- $\mathbf{V}$  is the value column matrix whose  $s^{th}$  component is  $V_\pi(s)$  -  $\mathbf{V}$  is the unknown variable

For a finite MDP, Bellman's equation admits an unique solution:

$$\mathbf{V} = (\mathbf{I} - \gamma \mathbf{P})^{-1} \mathbf{R}$$

where  $\mathbf{I}$  is the identity matrix. This requires  $(\mathbf{I} - \gamma \mathbf{P})$  to be invertible which ok since  $P$  is a probability matrix and  $\gamma < 1$  and of course to know  $\mathbf{P}$  and  $\mathbf{R}$ . Complexity  $O(N^3)$  ( $O(N^{2.807})$ ).

# The Value Function and the Q-Function

- The **optimal value function** is  $V^* : S \rightarrow \mathbb{R}$  such that

$$V^*(s) = \mathbb{E}[\sum_{t=1}^{\infty} \gamma^t r_t | s_1 = s]$$

- Observe (this is known as **Bellman's Equation**)

$$V^*(s) = \max_{a \in A} [\rho(s, a) + \gamma \mathbb{E}_{s' \sim \tau(s, a)} V^*(s')]$$

This differs from the previous equations because the max operator makes it non linear.

- The objective function in the above maximization problem is called the optimal action-value function, and is denoted by:

$$Q^*(s, a) = \rho(s, a) + \gamma \mathbb{E}_{s' \sim \tau(s, a)} V^*(s')$$

- corollary** The optimal policy is the **greedy policy** w.r.t.  $Q^*$ , namely  $\pi^*(s) = \operatorname{argmax}_a Q^*(s, a)$
- In particular, the optimal  $a_t$  is a deterministic function of  $s_t$ .

# Value Iteration

## Iterative Algorithm for finding $V^* - \epsilon$

Initialize  $\mathbf{V}_0$  to an arbitrary value;

Let  $t \leftarrow 0$ ;

**repeat**

for all  $s \in S$ ,  $V_{t+1}(s) = \max_{a \in A} [\rho(s, a) + \gamma \mathbb{E}_{s' \sim \mathcal{T}(s, a)} V_t(s')]$ ;  
 $t \leftarrow t + 1$ ;

**until**  $\|\mathbf{V}_t - \mathbf{V}_{t+1}\| < \frac{(1-\gamma)\epsilon}{\gamma}$ ;

- **Theorem**  $\|V_t - V^*\|_\infty \leq \gamma^t \|V_0 - V^*\|_\infty$   
 Convergence to  $V^*$  is ensured since  $0 \leq \gamma < 1$
- **Proof idea:**
  - Define  $T^* : \mathbb{R}^{|S|} \rightarrow \mathbb{R}^{|S|}$  to be the operator s.t.  $V_{t+1} = T^*(V_t)$
  - Show that  $T^*$  is a contraction mapping: for any two vectors in  $\mathbb{R}^{|S|}$  we have  $\|T^*(u) - T^*(v)\|_\infty \leq \gamma \|u - v\|_\infty$
  - The proof follows from (Banach's) fixed point theorem.

# Some Estimators

## Naive Learner

- ① Step 1: Estimate  $\tau$  and  $\rho$  by applying purely random policy
- ② Step 2: Apply Value Iteration to learn the optimal policy.

## Monte-Carlo simulation

In each state  $x$ , we simulate  $n$  trajectories  $((x_t^i)_{t \geq 0})_{1 \leq i \leq n}$  following policy  $\pi$ , where for any  $i = 1, \dots, n$ ,  $x_0^i = x$  and  $x_{t+1}^i \sim p(\cdot | x_t^i, \pi(x_t^i))$ .

We compute then:

$$\hat{V}_\pi(x) \simeq \frac{1}{n} \sum_{i=1}^n \sum_{t \geq 0} \gamma^t r(x_t^i, \pi(x_t^i))$$

Note: When we now  $\tau$  and  $\rho$ , finding  $\pi^*$  refers to planning, but when we can only observe sample trajectories, this is learning.

# Sample-based policy evaluation

The **Temporal Difference** method is based on stochastic approximation.

## TD(0)

```

 $V \leftarrow V_0$  //initialisation to any function;
for  $t = 0$  to  $T$  do
     $r' \leftarrow \rho(s, \pi(s))$  //reward;
     $s' \leftarrow \tau(s, \pi(s))$  //next state;
     $V(s) \leftarrow (1 - \alpha_t)V(s) + \alpha_t(r' + \gamma V(s'))$ ;
  
```

- Considering  $V(s) \leftarrow (1 - \alpha_t)V(s) + \alpha_t(r_{t+1} + \gamma V(s_{t+1}))$
- $r_{t+1} + \gamma V(s_{t+1})$  is the TD target
- $\alpha_t$  is the learning rate
- $r_{t+1} + \gamma V(s_{t+1}) - V(s_t)$  is the TD error

# An alternative - policy iteration: direct policy computing

```

 $\pi \leftarrow \pi_0$  (arbitrary policy);
 $\pi' \leftarrow \text{NIL}$ ;
while  $\pi \leq \pi'$  do
    Solve the linear equations
       $V_\pi(s) = \mathbb{E}[r|s, \pi(s)] + \gamma \sum_{s' \in S} \tau(s'|s, \pi(s)) V_\pi(s')$ ;
     $\mathbf{V} \leftarrow \mathbf{V}_\pi$  (policy evaluation, see slide A note on Bellman... );
     $\pi' \leftarrow \pi$ ;
    Improve the policy on each state;;
    for  $s \in S$  do
       $\pi(s) \leftarrow \arg \max_a [\mathbb{E}[r|s, a] + \gamma \sum_{s' \in S} \tau(s'|s, a) V_\pi(s')]$ ;
return  $\pi$ ;

```

This algorithm converges faster than value iteration one but each iteration requires the computation of a policy. It always return an optimal policy in at most  $|A|^{|S|}$  iterations.

Other possible approach: Linear programming.

# Q-Learning

- **Objective:** Estimate the optimal state-action value  $Q^*$  in the case of an unknown model
- Equivalent Bellman's equation for the  $Q$  function

$$Q^*(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim \tau(s, a)} \max_{a'} Q^*(s', a')$$

The optimal policy can be easily derived from  $Q^*$ :

$$\pi^*(s) = \operatorname{argmax}_{a \in A} Q^*(s, a) \text{ and } V^*(s) = \max_{a \in A} Q^*(s, a)$$

# Q-Learning

- Given  $(s_t, a_t, s_{t+1}, r_t)$  define  

$$\delta_{s_t, a_t}(Q) = Q(s_t, a_t) - (r_t + \gamma \max_{a'} Q(s_{t+1}, a'))$$
- Initialize  $Q_0$  and iteratively select a new state  $s_{t+1}$  and update

$$Q_{t+1}(s, a) = Q_t(s, a) - \alpha_t \delta_{s_t, a_t}(Q_t) 1_{[s=s_t, a=a_t]}$$

$$Q_{t+1}(s, a) = Q_t(s, a) + \alpha_t (r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t))$$

Can be viewed as a stochastic formulation of the value iteration algorithm presented before.

- Given a finite MDP, if for each pair  $(s, a)$  the  $\alpha_t \in [0, 1]$  verifies  $\sum_{t=0}^{\infty} \alpha_t(s, a) = \infty$  and  $\sum_{t=0}^{\infty} \alpha_t(s, a)^2 < \infty$ , Then the Q-learning algorithm converges to the optimal  $Q^*$ .  
 Condition on  $\alpha_t(s, a)$  impose that each state-action pair is visited infinitely many times



# Analysis of the formula

$$Q_{t+1}(s, a) = Q_t(s, a) + \alpha_t (r_t + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t))$$

- $Q_t(s, a)$ : Previous Q-value estimation
- $\alpha_t$ : learning rate
- $r_t$ : immediate reward
- $\gamma \max_{a'} Q(s_{t+1}, a')$ : Discounted estimate optimal Q-value of next state
- $Q(s_t, a_t)$ : Previous Q-value estimation

# Q-learning algorithm

```

 $Q \leftarrow Q_0$  (initialization, e.g. to 0);
for  $t \leftarrow 0$  to  $T$  do
     $s \leftarrow \text{SelectState}()$ ;
    for each step of epoch  $t$  do
         $a \leftarrow \text{SelectAction}(s, \pi)$  - from policy  $\pi$  deduced from  $Q$  ;
         $r' \leftarrow \text{Reward}(s, a)$ ;
         $s' \leftarrow \text{NextState}(s, a)$ ;
         $Q(s, a) \leftarrow Q(s, a) + \alpha[r' + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ ;
         $s \leftarrow s'$ ;
return  $Q$ ;

```

# Exploration for Q-Learning

- Q-Learning can be applied for any choice of  $a_t$  (it is an “off policy” learner)
- Q-Learning converges to the optimal state value function  $Q_*$
- Speed of convergence can be improved if we balance the exploration-exploitation tradeoff (by one of the methods described previously)
- The convergence of the “tabular” Q-learning (namely, maintaining  $Q$  is a table of size  $|S| \times |A|$ ) becomes very slow.
- We describe two approaches to overcome this problem:
  - Function Approximation
  - Temporal Abstractions

# Random Alternative SARSA

## State-Action-Reward-State-Action - on-policy

```

 $Q \leftarrow Q_0$  (initialization, e.g. to 0);
 $s \leftarrow s_0$ ;
 $a \leftarrow \text{SelectAction}(\pi(Q), s)$ ;
for  $t \leftarrow 0$  to  $T$  do
     $s \leftarrow \text{SelectState}()$ ;
    for each step of epoch  $t$  do
         $r' \leftarrow \text{RandomReward}(s, a)$ ;
         $s' \leftarrow \text{NextState}(s, a)$ ;
         $a' \leftarrow \text{SelectAction}(s', \pi(Q))$  - from policy  $\pi$  deduced from  $Q$  ;
         $Q(s, a) \leftarrow Q(s, a) + \alpha[r' + \gamma Q(s', a') - Q(s, a)]$ ;
         $s \leftarrow s'$ ;
return  $Q$ ;
  
```

# Function Approximation for Q-Learning

- Do state aggregation
- Maintain a parametric hypothesis class of  $Q$  functions
- Rewrite  $\delta$  as a function of the parameter  $\theta$ :

$$\delta_{s_t, a_t}(\theta) = Q_\theta(s_t, a_t) - (r_t + \gamma \max_{a'} Q_{\theta_t}(s_{t+1}, a'))$$

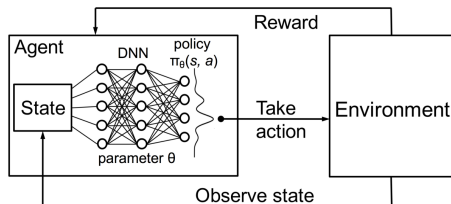
- Since we want to minimize  $\frac{1}{2}\delta_{s_t, a_t}(\theta_t)$  we take a gradient step:

$$\theta_{t+1} = \theta_t - \eta_t \delta_{s_t, a_t}(\theta_t) \nabla Q_\theta(s_t, a_t)$$

we can use any function approximator (e.g. a neural network), maybe a bit unstable and few guarantees of convergence - can do updates in batch and not at each step.

# Deep-Q-Learning

- Used first by DeepMind to learn to play Atari games
- Let  $Q_\theta : S \rightarrow \mathbb{R}^{|A|}$  be a deep network, where we take  $S \subset \mathbb{R}^d$  and assume that  $|A|$  is not too large
- Exploration:  $\epsilon$ -greedy (choose best action -  $\max Q_t(a, s)$  - with proba  $(1 - \epsilon)$  (exploitation) and random action with proba  $\epsilon$  (exploration))
- **Memory replay:** After executing  $a_t$  and observing  $r_t, s_{t+1}$  we store the example  $(s_t, a_t, r_t, s_{t+1})$  in a database.  
Instead of updating just based on the last example, update based on a mini-batch of random examples from the database.



# Deep-Q-Learning (2)

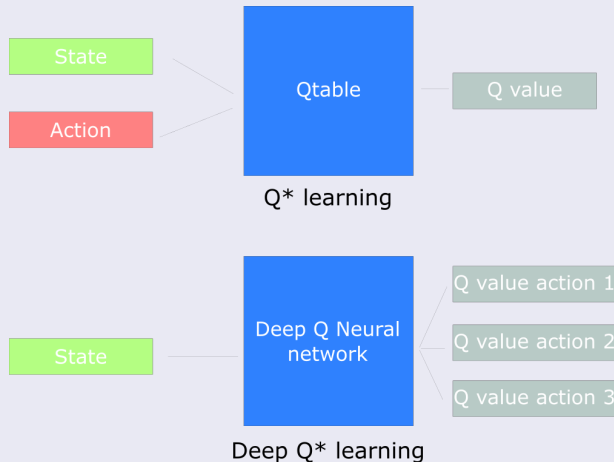
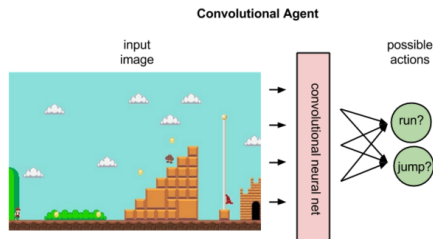


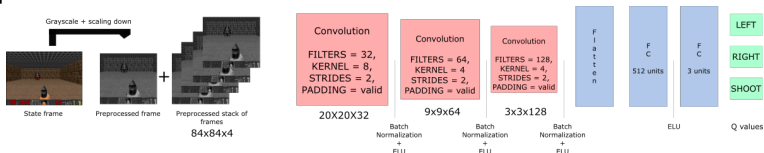
Figure 1: Difference between Classic Q-learning and Deep-Q-Learning

# Deep-Q-Learning (3)

- Example 1 from <https://skymind.ai/wiki/deep-reinforcement-learning>



- Example 2



<https://medium.freecodecamp.org/an-introduction-to-deep-q-learning-lets-play-doom-54d02d8017d8>



# Intuition: Structuring a State Space

- Consider some state space  $S \subset \mathbb{R}^d$
- Suppose we **partition** it to  $S = S_1 \cup S_2 \cup \dots \cup S_k$
- Assuming homogenous actions within each  $S_i$ , we can apply  $Q$ -learning while using  $[k]$  as a new state space
- One can think of Deep-Q-Learning as automatically finding the partition (the first layers of the network)

# Other solution: Temporal Abstraction

- Decisions are often structured into sub-tasks with a broad range of time scale, *i.e.*:
  - Task: call a taxi
  - Step 1 finding my phone
  - Step 2: finding the number
  - Step 3: dialing the first digit
  - ...
  - Step 20: commanding my finger muscle to move into the right place ...
- **Options** [Sutton, Precup, Singh]
  - An option is a pair  $(\pi, \beta)$  where
    - $\pi : S \rightarrow A$  is the policy to apply within the “option”
    - $\beta : S \rightarrow [0, 1]$  is a stochastic termination function
  - Instead of directly choosing actions, the agent picks an option  $o_t \in O$ , and this option is applied until it terminates.
  - That is, we should learn a policy over options,  $\mu : S \rightarrow O$
  - We can learn  $\mu$  similarly to how we learn a vanilla policy, and the advantage is that it may be easier to pick  $O$  than picking  $A$

# Monte-Carlo Tree Search

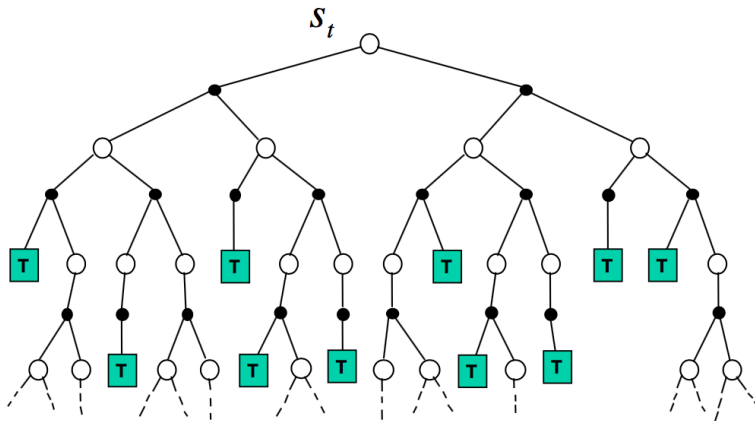
# Simple Monte-Carlo Search

- Given a model and simulation policy
- For each action  $a \in A$ 
  - Simulate  $K$  episodes from current (real) state  $s_t$   
 $\{s_t, a, R_{t+1}^k, \dots, S_T^k\}$
  - Evaluate actions by computing the mean reward  
 $Q(s_t, a) = \frac{1}{K} \sum_{k=1}^K G_{k,t}$   
 where  $G_{k,t}$  is the sum of (discounted) reward for episode  $k$ .
  - Select current (real) action with maximum value

$$a_t = \operatorname{argmax}_{a \in A} Q(s_t, a)$$

# Can We do better than 1-step?

- Forward search algorithms select the best actions by lookahead
- Build a search tree from current state
- No need to solve the full MDP



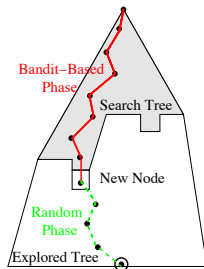
# Monte-Carlo Tree Search (MCTS)

- Given a model (you can think of a game)
- Build a search tree rooted at current state  $s_t$
- Sample actions and next states using Exploration-Exploitation strategy
- Iteratively construct and update tree by performing K simulation episodes starting from the root state
- After search is finished, select current (real) action with maximum value in search tree.

$$a_t = \operatorname{argmax}_{a \in A} Q(s_t, a)$$

# Basic Procedure

- Start from root node/initial state
- Step 1
  - Visit the tree according to an exploration-exploitation strategy using a (multi-armed)-bandit approach according to the current policy until reaching a leaf
- Step 2
  - Simulate next action by a (simple) random selection of action/state (e.g. uniform) until reaching a final state or a fixed number of rollouts.
- Update information in visited nodes (Propagation)



# Upper Confidence Tree (UCT) Search

- Borrow idea from UCB
- Maintain an upper confidence bound over reward of each arm at each node  $\hat{Q}_B(s, a, i) = \mu_{i,a} + c \sqrt{\frac{O(\log N(i))}{N(i,a)}}$ 
  - $c$ : controls the exploration-exploitation trade-off
  - $N(i)$ : number of times node  $i$  is visited
  - $(N(i, a))$ : number of times arm  $a$  is selected at node  $i$ .
- When iterating, at node  $i$  select action/arm with highest upper bound to simulate and expand (or evaluate) in tree  $\operatorname{argmax}_a \hat{Q}_B(s, a, i)$
- The policy used for simulation can change across each episode



# Advantages

- Allow to deal with high dimensional problems (e.g. game of go)
- Highly selective best first search
- Evaluated states dynamically
- Uses sampling to break the curse of dimensionality
- Works for "black-box" models (only require samples)
- Computationally efficient, anytime, parallelisable
- Variants modify the sampling strategy, exploration-exploitation approach, use heuristics that may depend on the target game

# Limitations of MDPs

- Other Reinforcement learning algorithms
  - SARSA (State Action Reward' State' Action'): very similar to Q-learning except that the action for the next state is selected by the policy instead of using a max
  - $TD(\lambda)$ : takes into account the reward over multiple steps ahead, the policy is evaluated directly without considering the different actions as in the linear Bellman equations presented before.
- The Markovian assumption is mathematically convenient but rarely holds in practice.
- **POMDP = Partially Observed MDP**: There is a hidden Markovian state, but we only observe a view that depends on it.
- Another approach is “direct policy search”, that does not necessarily rely on the Markovian assumption.

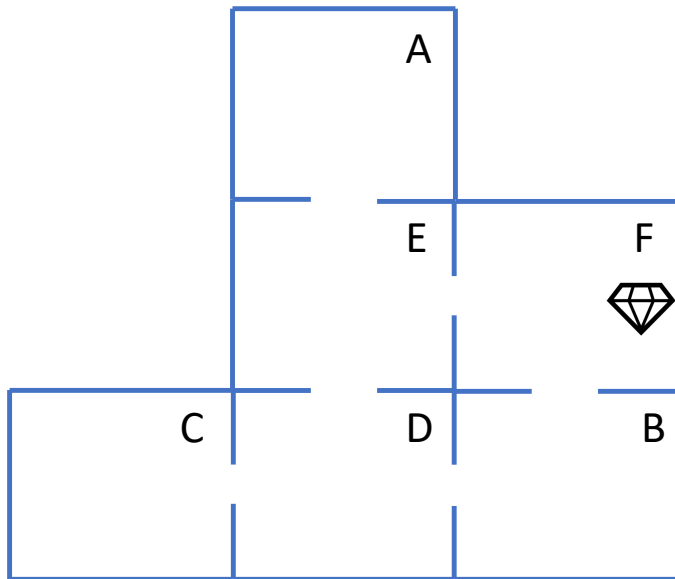
# An example

## Context

- We consider an environment composed of 6 rooms:  $A, B, C, D, E, F$ . Some rooms are only accessible from other rooms (you can think to a maze for example)
- The objective is to learn how to go in room  $F$  from other rooms.
- Each room is represented by a state.
- The possible actions consist to move from one room to another.

Example taken from: Kardi Teknomo, 2005. Q-Learning by Examples.  
<http://people.revoledu.com/kardi/tutorial/ReinforcementLearning/index.html>

# Illustration of the environment



# Reinforcement learning formulation

## Modeling

- We use  $\gamma = 0.8$  as discounted factor.
- We use  $\eta = 1$  as learning parameter (the same for all actions/states) to simplify.
- We will use a pure exploration strategy.
- We draw an Initial state at random, say room  $B$ .
- $Q$  is initialized as a **0** matrix:

| state \ state | A | B | C | D | E | F |
|---------------|---|---|---|---|---|---|
| A             | 0 | 0 | 0 | 0 | 0 | 0 |
| B             | 0 | 0 | 0 | 0 | 0 | 0 |
| C             | 0 | 0 | 0 | 0 | 0 | 0 |
| D             | 0 | 0 | 0 | 0 | 0 | 0 |
| E             | 0 | 0 | 0 | 0 | 0 | 0 |
| F             | 0 | 0 | 0 | 0 | 0 | 0 |

# Reinforcement learning formulation

## Modeling

- Reward matrix:

| state \ actions | A | B | C | D | E | F |
|-----------------|---|---|---|---|---|---|
| A               | - | - | - | - | 0 | - |
| B               | - | - | - | 0 | - | 1 |
| C               | - | - | - | 0 | - | - |
| D               | - | 0 | 0 | - | 0 | - |
| E               | 0 | - | - | 0 | - | 1 |
| F               | - | 0 | - | - | 0 | 1 |

\*A '-' indicates that the move action is not possible,

\*1 indicates the goal,

\*for 0 values, we have to evaluate their interest when looking for the best policy.

# Episode 1

- Let us start from state  $B$ . Looking at the reward matrix, we have two possible actions: go to state  $D$  or go to state  $F$ . Imagine that we choose  $F$  by random selection.
- Now, we are in state  $F$  and we have 3 possible actions: go to state  $B$ ,  $E$  or  $F$ .

- Recall

$$Q(s_t, a_t) = Q(s_t, a_t) + \eta(R(s_t, a_t) + \gamma \max_{a'}[Q(s_{t+1}, a')] - Q(s_t, a_t)),$$

- Since  $\eta = 1$ , then

$$Q(s_t, a_t) = (R(s_t, a_t) + \gamma \max_{a'}[Q(s_{t+1}, a')]) , \text{ so}$$

- $Q(B, F) = (R(B, F) + 0.8 \max[Q(F, B), Q(F, E), Q(F, F)]) = 1 + 0.8 * 0 = 1$

At this point since  $Q$  is the **0** matrix,  $Q(F, B)$ ,  $Q(F, E)$ ,  $Q(F, F)$  are all zero.

- We then decide to move to  $F$  which becomes the current state.

# Episode 1 (end)

- Since  $F$  is the goal state, we finish the episode and the updated  $Q$  matrix is:

| state \ state | A | B | C | D | E | F |
|---------------|---|---|---|---|---|---|
| A             | 0 | 0 | 0 | 0 | 0 | 0 |
| B             | 0 | 0 | 0 | 0 | 0 | 1 |
| C             | 0 | 0 | 0 | 0 | 0 | 0 |
| D             | 0 | 0 | 0 | 0 | 0 | 0 |
| E             | 0 | 0 | 0 | 0 | 0 | 0 |
| F             | 0 | 0 | 0 | 0 | 0 | 0 |



# Episode 2

- We start with initial random state, say  $D$ .
- From matrix  $R$  we have three possible actions:  $B$ ,  $C$  and  $E$ . By random selection, assume we select  $B$  as initial state. No update on  $Q$  since all the rewards are 0 here.
- Now, imagine we have moved to state  $B$ , we have 2 possible actions  $D$  or  $F$ .
- We have now
$$Q(D, B) = (R(D, B) + 0.8 * \max[Q(B, D), Q(B, F)]) = 0 + 0.8 = .8$$
from the updated matrix computed before.

# Episode 2

- We now update the  $Q$  matrix:

| state \ state | A | B   | C | D | E | F |
|---------------|---|-----|---|---|---|---|
| A             | 0 | 0   | 0 | 0 | 0 | 0 |
| B             | 0 | 0   | 0 | 0 | 0 | 1 |
| C             | 0 | 0   | 0 | 0 | 0 | 0 |
| D             | 0 | 0.8 | 0 | 0 | 0 | 0 |
| E             | 0 | 0   | 0 | 0 | 0 | 0 |
| F             | 0 | 0   | 0 | 0 | 0 | 0 |

- We move to state  $B$ . This state is not the final one, we have to continue.

# Episode 2, continued

- Now, the current state is  $B$ . There are 2 possible actions: go to  $D$  or go to  $F$ .
- By lucky draw, we select state  $F$ . From  $F$ , we have 3 possible actions: go to  $B$ ,  $E$  or  $F$ .
- We compute the  $Q$  value,  

$$Q(B, F) = (R(B, F) + 0.8 * \max[Q(F, B), Q(F, E), Q(F, F)]) = 1 + 0.8 * 0 = 1.$$
 $Q(F, B)$ ,  $Q(F, E)$  and  $Q(F, F)$  are all zeros.  
 The  $Q$  matrix does not need to be updated.
- $F$  is the goal state, the episode is finished.

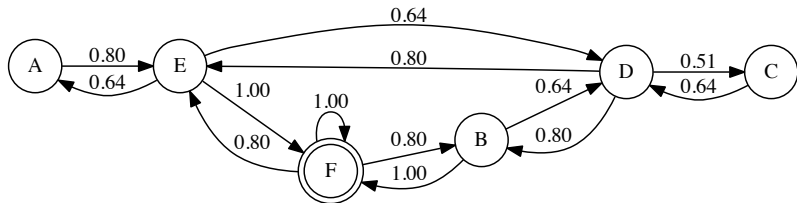
# Continuing

- By getting more and more information by exploring the state/action space, the values of the  $Q$  matrix will converge to:

| state \ state | A    | B    | C    | D    | E    | F    |
|---------------|------|------|------|------|------|------|
| A             | 0    | 0    | 0    | 0    | 0.80 | 0    |
| B             | 0    | 0    | 0    | 0.64 | 0    | 1.00 |
| C             | 0    | 0    | 0    | 0.64 | 0    | 0    |
| D             | 0    | 0.80 | 0.51 | 0    | 0.80 | 0    |
| E             | 0.64 | 0    | 0    | 0.64 | 0    | 1.00 |
| F             | 0    | 0.80 | 0    | 0    | 0.80 | 1.00 |

# Extraction of a policy

- The informations obtained from the  $Q$  matrix can be summarized by the following automaton



- Imagine we start in state  $C$ , the agent can extract the following policy from the  $Q$  matrix:  
 $C \rightarrow D \rightarrow B \rightarrow F$
- Agent is able to extract the best policy at convergence (or close to).

# Conclusion

# Conclusion

- Reinforcement Learning is a powerful and useful learning setting, but is much harder than Supervised Learning
- The Exploration-Exploitation Tradeoff
- MDP: Connecting the future rewards to current actions using a Markovian assumption
- Importance of having efficient resolution (Dynamic programming, stochastic processes (Monte Carlo), ...)
- Monte Carlo Tree Search (MCTS) - AlphaGo / AlphaZero
- Combination with Deep Learning  
<https://www.youtube.com/watch?v=n7370dzCZ1o>  
<https://www.youtube.com/watch?v=79pmNdyxEGo>
- Bandits (MDP with only 1 state)